



《人工智能与Python程序设计》— 知识点摘要



人工智能与Python程序设计 教研组



Python和Numpy进阶 知识补充

提纲



- ☐ Python基础
- ☐ NumPy与PyTorch
- ☐ 人工智能模型

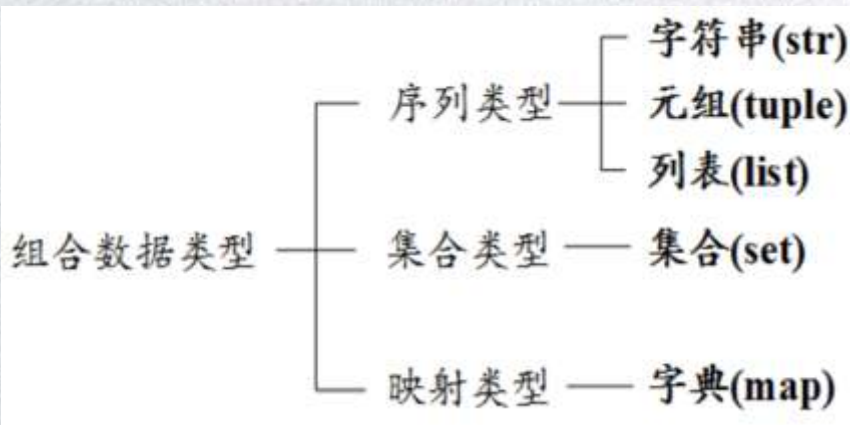


组合数据类型



组合数据类型概述

- 组合数据类型能够将多个同类型或不同类型的数据组织起来，通过单一的表达使数据操作更有序更容易。
- 根据数据之间的关系，组合数据类型可以分为三类：序列类型、集合类型和映射类型。





序列类型

- Python语言中有很多数据类型都是序列类型，其中比较重要的是：
str（字符串）、tuple（元组）、list（列表）
 - 元组是包含0个或多个数据项的不可变序列类型。元组生成后是固定的，其中任何数据项不能替换或删除。
 - 列表则是一个可以修改数据项的序列类型，使用也最灵活。
- 索引体系：



序列类型

- 序列类型通用操作符和函数

操作符	描述
<code>x in s</code>	如果x是s的元素，返回True，否则返回False
<code>x not in s</code>	如果x不是s的元素，返回True，否则返回False
<code>s + t</code>	连接s和t
<code>s * n</code> 或 <code>n * s</code>	将序列s复制n次
<code>s[i]</code>	索引，返回序列的第i个元素
<code>s[i: j]</code>	分片，返回包含序列s第i到j个元素的子序列（不包含第j个元素）
<code>s[i: j: k]</code>	步骤分片，返回包含序列s第i到j个元素以k为步数的子序列
<code>len(s)</code>	序列s的元素个数（长度）
<code>min(s)</code>	序列s中的最小元素
<code>max(s)</code>	序列s中的最大元素
<code>s.index(x[, i[, j]])</code>	序列s中从i开始到j位置中第一次出现元素x的位置
<code>s.count(x)</code>	序列s中出现x的总次数



集合类型

- 集合类型与数学中集合的概念一致，即包含0个或多个数据项的无序组合。集合中元素不可重复，元素类型只能是固定数据类型，例如：整数、浮点数、字符串、元组等。列表、字典和集合类型本身都是可变数据类型，不能作为集合的元素出现。
- 由于集合是无序组合，它没有索引和位置的概念，不能分片，集合中元素可以动态增加或删除。集合用大括号（{}）表示，可以用赋值语句生成一个集合。



集合类型

- 由于集合元素是无序的，集合的打印效果与定义顺序可以不一致。由于集合元素独一无二，使用集合类型能够过滤掉重复元素。set(x)函数可以用于生成集合。

```
In [7]: S = {"信息楼", "RUC", (12, "AI"), 100872}
S
Out[7]: {(12, 'AI'), 100872, 'RUC', '信息楼'}

In [8]: S = {"信息楼", "RUC", (12, "AI"), 100872, "RUC"}
S
Out[8]: {(12, 'AI'), 100872, 'RUC', '信息楼'}

In [10]: T = set("intelligence")
T
Out[10]: ['c', 'e', 'g', 'i', 'l', 'n', 't']

In [11]: T = set(("python", "learning", "artificial", "intelligence"))
T
Out[11]: ('artificial', 'intelligence', 'learning', 'python')
```




集合类型

- 集合类型有10个操作函数或方法

函数或方法	描述
<code>S.add(x)</code>	如果数据项 <code>x</code> 不在集合 <code>S</code> 中，将 <code>x</code> 增加到 <code>s</code>
<code>S.clear()</code>	移除 <code>S</code> 中所有数据项
<code>S.copy()</code>	返回集合 <code>S</code> 的一个拷贝
<code>S.pop()</code>	随机返回集合 <code>S</code> 中的一个元素，如果 <code>S</code> 为空，产生 <code>KeyError</code> 异常
<code>S.discard(x)</code>	如果 <code>x</code> 在集合 <code>S</code> 中，移除该元素；如果 <code>x</code> 不在，不报错
<code>S.remove(x)</code>	如果 <code>x</code> 在集合 <code>S</code> 中，移除该元素；不在产生 <code>KeyError</code> 异常
<code>S.isdisjoint(T)</code>	如果集合 <code>S</code> 与 <code>T</code> 没有相同元素，返回 <code>True</code>
<code>len(S)</code>	返回集合 <code>S</code> 元素个数
<code>x in S</code>	如果 <code>x</code> 是 <code>S</code> 的元素，返回 <code>True</code> ，否则返回 <code>False</code>
<code>x not in S</code>	如果 <code>x</code> 不是 <code>S</code> 的元素，返回 <code>True</code> ，否则返回 <code>False</code>



字典类型的概念

- Python语言中我们可以使用字典（dict）实现映射
 - 字典是包含0个或者多个键值对信息的关联数组
 - 关联数组是支持以下操作的抽象数据类型：
 - 向关联数组添加键值对
 - 从关联数组内删除键值对
 - 修改关联数组内的键值对
 - 根据已知的键寻找键值对
 - 注意：字典中的键是唯一的，无法保存一对多的映射
 - "RUC"=>"人民大学"
 - "RUC"=>"中国人民大学"
 - 解决方法"RUC"=>["人民大学", "中国人民大学"]

简称（键）	全称（值）
RUC	中国人民大学
THU	清华大学
PKU	北京大学
BIT	北京理工大学
.....	



字典类型的创建

- 字典类型的创建方法有以下几种：
 - 使用{}创建

```
[2]: dict1 = {'RUC': '中国人民大学',  
             'THU': '清华大学',  
             'PKU': '北京大学',  
             'BIT': '北京理工大学',  
             'BUAA': '北京航空航天大学'} # 用{}创建  
empty_dict = {} # 可以用这个方式创建空字典  
  
print(dict1)  
print(empty_dict)
```

```
{'RUC': '中国人民大学', 'THU': '清华大学', 'PKU': '北京大学', 'BIT': '北京理工大学', 'BUAA': '北京航空  
航天大学'}  
{}
```




字典类型的创建

- Python语言提供了一种基于字典的灵活的给函数传递参数的方式
 - 不需在定义函数的时候指定参数的个数和名称
 - 参数会以字典的形式被传递到函数内

```
[6]: def g(**kwargs):  
      print(type(kwargs))  
      print(kwargs)  
  
      g()  
      g(RUC='中国人民大学', THU='清华大学')  
  
<class 'dict'>  
{}  
<class 'dict'>  
{'RUC': '中国人民大学', 'THU': '清华大学'}
```



字典类型的操作

- 利用键索引字典中保存的值：
 - 可以使用get()方法来访问字典中保存的值

```
[8]: print(dict1.get('RUC')) # 也可以使用get方法  
      print(dict1.get('FDU', '名称未知')) # get方法的第二个参数是若键信息不在字典里时返回的默认值信息
```

```
中国人民大学  
名称未知
```

```
[9]: print('RUC' in dict1)  
      print('FDU' in dict1)
```

```
True  
False
```



字典类型的操作

- 通过键增加、修改值信息和删除相应的键值对：
 - 可以使用[]来增加、修改和删除字典中保存的信息

```
[10]: dict2 = dict(dict1)
      print(dict2)
      dict2['FUDU'] = '复旦大学'
      dict2['RUC'] = 'Renmin University of China'
      print(dict2)
```

```
{'RUC': '中国人民大学', 'THU': '清华大学', 'PKU': '北京大学', 'BIT': '北京理工大学', 'BUAA': '北京航空航天大学'}
```

```
{'RUC': 'Renmin University of China', 'THU': '清华大学', 'PKU': '北京大学', 'BIT': '北京理工大学', 'BUAA': '北京航空航天大学', 'FUDU': '复旦大学'}
```

```
[11]: dict2 = dict(dict1)
      print(dict2)
      del dict2['RUC']
      print(dict2)
```

```
{'RUC': '中国人民大学', 'THU': '清华大学', 'PKU': '北京大学', 'BIT': '北京理工大学', 'BUAA': '北京航空航天大学'}
```

```
{'THU': '清华大学', 'PKU': '北京大学', 'BIT': '北京理工大学', 'BUAA': '北京航空航天大学'}
```


字典类型的操作

- 获取一个字典中的所有键、值、以及键值对：
 - `<d>.keys()`, `<d>.values()`, `<d>.items()`

```
[13]: dict2 = dict(dict1)
      keys = dict2.keys()
      print(keys)
      values = dict2.values()
      print(values)
      items = dict2.items()
      print(items)

dict_keys(['RUC', 'THU', 'PKU', 'BIT', 'BUAA'])
dict_values(['中国人民大学', '清华大学', '北京大学', '北京理工大学', '北京航空航天大学'])
dict_items([('RUC', '中国人民大学'), ('THU', '清华大学'), ('PKU', '北京大学'), ('BIT', '北京理工大学'), ('BUAA', '北京航空航天大学')])
```



字典类型的操作

- 使用for ... in ... 语句遍历字典

```
[15]: dict2 = dict(dict1)
      for key in dict2: # 该语句会遍历字典中所有键
          print(key)

      for key, value in dict2.items(): # 这样可以遍历所有键值对
          print(key, value)
```

```
RUC
THU
PKU
BIT
BUAA
RUC 中国人民大学
THU 清华大学
PKU 北京大学
BIT 北京理工大学
BUAA 北京航空航天大学
```



词频统计-字典方法

- 字典方法词频统计：

```
[25]: import jieba
txt = open('./高瓴人工智能学院简介.txt', 'r', encoding='utf-8').read()
words = jieba.lcut(txt)
counts = {}
for word in words:
    if len(word) == 1: #排除单个字符的分词结果
        continue
    else:
        counts[word] = counts.get(word, 0) + 1
items = list(counts.items())
items.sort(key=lambda x: x[1], reverse=True)
for i in range(15):
    word, count = items[i]
    print('{0:<10}{1:>5}'.format(word, count))
```

人工智能	13
学院	8
一流	5
中国人民大学	4
未来	3
高瓴	3
院长	3
打造	3
全球	3
研究	3
联合	3
时代	2
影响	2
技术	2
发展	2



函数参数



可选参数

- 在定义函数时，有些参数可以存在默认值
- 函数被调用时，如果没有传入对应的参数值，则使用函数定义时的默认值代替
- 可选参数必须定义在非可选参数后面

```
In [6]: def dup(str, times=2):  
        print(str*times)  
        dup("ruc~")
```

ruc~ruc~

```
In [7]: dup("ruc!", 4)
```

ruc!ruc!ruc!ruc!



可变数量参数

- 在函数定义时，可以设计可变数量参数，通过参数前增加星号 (*) 实现
- 可变参数只能出现在参数列表的最后
 - 可变参数被当作元组类型传入函数中

```
In [9]: def vfunc(a, *b):  
        print(type(b))  
        for n in b:  
            a += n  
        return a  
vfunc(1, 2, 3, 4, 5)
```

```
<class 'tuple'>
```

```
Out[9]: 15
```




参数的位置和名称传递

- 实参默认采用按照位置顺序的方式传递给函数
 - *def func(x1,y1,z1,x2,y2,z2)*
 - *func(1,2,3,4,5,6)*
- Python提供了按照形参名称输入实参的方式，调用如下：
 - *result = func(x2=4, y2=5, z2=6, x1=1, y1=2, z1=3)*
- 由于调用函数时指定了参数名称，所以参数之间的顺序可以任意调整



字典类型的创建

- Python语言提供了一种基于字典的灵活的给函数传递参数的方式
 - 不需在定义函数的时候指定参数的个数和名称
 - 参数会以字典的形式被传递到函数内

```
[6]: def g(**kwargs):  
      print(type(kwargs))  
      print(kwargs)  
  
      g()  
      g(RUC='中国人民大学', THU='清华大学')  
  
<class 'dict'>  
{}  
<class 'dict'>  
{'RUC': '中国人民大学', 'THU': '清华大学'}
```



IO编程



文件概述

- 二进制文件

- 由比特1和比特0所组成，没有统一的字符编码，文件内部数据的组织格式和文件用途有关。
- 二进制文件与文本文件的主要区别在于是否有统一的字符编码。
- 二进制是信息按照非字符但特定格式形成的文件，数字图像的JPEG和PNG，数字音频的MP3和WAV等。
- 由于二进制文件没有统一的字符编码，故只能当做字节流，而不能看做是字符串。



文本文件

- 文本文件的字符编码

- 编码是信息从一种形式转换为另一种形式的过程
- ASCII编码、Unicode编码、UTF-8编码等
- Unicode编码：
 - 跨语言、跨平台进行文本转换和处理
 - 对每种语言中字符设定统一且唯一的二进制编码
 - 每个字符两个字节长
 - 基本的Unicode字符有两个字节长，有65536个编码空间
- UTF-8编码
 - 可变长度的Unicode的实现方式





文件读写

- 文件操作
 - 文件读写是最常见的IO操作，遵循**打开-操作-关闭**步骤。
 - Python内置了读写文件的函数，请求操作系统打开一个文件对象
- 文件的打开
 - 利用Python内置的open()函数，以读文件的模式打开一个文件对象
 - <name>：文件路径及名称
 - <mode>：打开模式
 - <variable>：文件对象名

```
<variable> = open(<name>, <mode>)
```




文件读写

- 文件的写入
 - 从计算机内存向文件写入数据
 - Python提供3个与文件内容写入有关的方法

操作方法	含义
<code><variable>.write()</code>	向文件写入一个字符串或字节流
<code><variable>.writelines()</code>	将一个元素全为字符串的列表写入文件
<code><variable>.seek(offset)</code>	改变当前文件操作指针的位置， <code>offset</code> 的值： 0--文件开头； 1--当前位置； 2--文件结尾



文件读写

- with ... as ...用法

- 文件的输入输出、数据库的连接断开等，都是很常见的资源管理操作。但资源都是有限的，在写程序时，必须保证这些资源在使用过后得到释放，不然就容易造成资源泄露，轻者使得系统处理缓慢，严重时会使系统崩溃。
- Python使用 with as 语句操作上下文管理器，它能够帮助我们自动分配并且释放资源。

with 表达式 [as target]:
 代码块



os模块编程

- 操作系统层级
 - 命令行下，可以通过输入操作系统提供的各种命令实现文件、目录操作
- Python
 - Python内置的os模块可以直接调用操作系统提供的接口函数。
 - os模块提供了多数操作系统的功能接口函数。当os模块被导入后，它会自适应于不同的操作系统平台，根据不同的平台进行相应的操作。
- 目录操作
 - 路径操作一部分在os.path模块中，一部分在os模块中
 - 查看当前的绝对路径：os.path.abspath()
 - 多个目录路径合并：os.path.join()
 - 在Linux/Unix/Mac下，os.path.join()返回拼接 '/' ；在windows下返回拼接 '\\'



类与实例



类和实例

- 类和实例是OOP中最为重要的概念
 - 类是抽象的模板，比如Student 类，
 - 实例是根据类创建出来的一个个具体的“对象”，每个对象都拥有相同的方法，但各自的数据可能不同。

```
Michael = Student('Michael', 98) # 创建实例对象
Kristen = Student('Kristen', 93) # 创建实例对象

Michael.print_score() # 调用类的方法
Kristen.print_score() # 调用类的方法

Michael: 98
Kristen: 93
```



类和实例

- 类和实例是OOP中最为重要的概念
 - 类是抽象的模板，比如Student 类，
 - 实例是根据类创建出来的一个个具体的“对象”，每个对象都拥有相同的方法，但各自的数据可能不同。

```
Michael = Student('Michael', 98) # 创建实例对象
Kristen = Student('Kristen', 93) # 创建实例对象

Michael.print_score() # 调用类的方法
Kristen.print_score() # 调用类的方法

Michael: 98
Kristen: 93
```




类和实例

- 类起到模板的作用，可以在创建实例的时候，把一些我们认为必须绑定的属性强制填写进去。
- 通过定义一个特殊的__init__方法（构造函数），在创建实例的时候，就把name, score等属性绑上去
 - __init__前后分别有两根下划线
 - __init__()的第一个参数永远是self，表示创建的实例本身。因此，在__init__()内部，就可把各种属性绑定到self，因为self就指向创建的实例本身。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score
```



数据封装

- Student 的实例本身就拥有这些数据，故要访问它们，可以直接在 Student 类的内部定义访问数据的函数，进而把“数据”封装。
- 封装数据的函数是和Student类本身是关联起来的，我们称之为**类的方法**。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score  
  
    def print_score(self):  
        # 类的方法  
        print('%s: %d' % (self.name, self.score))
```



继承与多态

- 在OOP 中定义一个class 时，可以从某个现有的class继承，新的class 称为子类(subclass)，而被继承的class 称为基类、父类或超类(base class, super class)。
- 继承：一个派生类(derived class)继承基类的字段和方法，即子类获得了父类的全部功能。
- 举例：Undergraduate类继承Student类， Dog类继承Animal类

```
class Animal(object):  
    def run(self):  
        print('Animal is running...')
```




继承与多态

- 对于Dog来说，Animal就是它的父类，对于Animal来说，Dog就是它的子类。
- 由于Animal实现了run()方法，Dog和Cat也拥有了run()方法。

```
class Dog(Animal):  
    pass
```

```
class Cat(Animal):  
    pass
```

```
dog = Dog()  
dog.run()
```

```
cat = Cat()  
cat.run()
```

```
Animal is running...  
Animal is running...
```

继承与多态

- 在继承时，也可对子类增添新的方法，如

```
class Dog(Animal):  
  
    def run(self):  
        print('Dog is running...')
```

```
    def eat(self):  
        print('Eating meat...')
```

```
dog = Dog()  
dog.run()
```

```
Dog is running...
```

子类的run()覆盖了父类的run()方法，
即多态性

添加了新的子类方法



继承与多态

- 多态：为不同数据类型的实体提供统一的接口。
 - 简单而言：相同的消息给予不同的对象会引发不同的动作，如前面的Animal类和Dog类在处理run()方法上动作的差异。



继承与多态

- 进一步理解多态
 - 定义一个class，实际上就定义了一种新的数据类型，类似于Python内置数据类型

```
a = list() # a是list类型  
b = Animal() # b是Animal类型  
c = Dog() # c是Dog类型
```



理解PYTHON语言



变量引用对象

- 变量赋值：本质上是将变量“贴”在对象上，运行使用变量来访问、修改对象
 - 对象是一块内存空间，内存空间里存储它们所表示的值；
 - 变量是到内存空间的一个**标签或引用**，也就是拥有指向对象存储的空间；
 - 引用就是自动形成的从变量到对象的映射关系（类似指针）
 - 引用可以看成对象的别名，通过别名可以直接操纵对象
 - 但与C语言中的指针不同，我们无法直接修改指针的值
 - 只能通过变量访问对象，或通过赋值更改变量指向的对象
 - **赋值**是将一个**变量标签**与一个**实际对象**建立关联的过程



可变对象和不可变对象

- 可变对象可以被修改，包括列表list、字典dict、集合set
 - 给出一些例子？
- 不可变对象无法修改，包括数字、字符串str，元组tuple
 - 给出一些例子？



可变对象和不可变对象

- 不可变对象发生修改：

```
name="Python"  
name+="AI"
```

- 不可变对象：
 - 第一句创建一个字符串对象，并让变量name引用该对象。
 - 按照C++中的理解，第二句试图修改name这个字符串。
 - 但是在python中，其实新建了一个值为" PythonAI" 的字符串对象，并让name引用该对象。

```
>>> name = 'python'  
>>> id(name)  
39862128  
>>> name2 = name  
>>> print(name2)  
python  
>>> id(name2)  
39862128  
>>> name += ' AI'  
>>> print(name)  
python AI  
>>> id(name)  
39866416  
>>> id(name2)  
39862128  
>>> print(name2)  
python  
>>>
```

如何验证这一情况？



可变对象和不可变对象

- 可变与不可变指的是**顶层对象**不可变
 - 不可变是指所包含对象的标识符不能发生改变
 - 也就是每个元素对应的id值不动
 - 但是如果元素本身可以被修改，则不限制

```
>>> a = (1, 3, [])  
>>> a[1] = 2  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'tuple' object does not support item assignment  
>>>
```

```
>>> a = (1, 3, [])  
>>> [id(x) for x in a]  
[8790970992304, 8790970992368, 39823360]  
>>> a[-1].append(1)  
>>> [id(x) for x in a]  
[8790970992304, 8790970992368, 39823360]  
>>>
```


浅复制

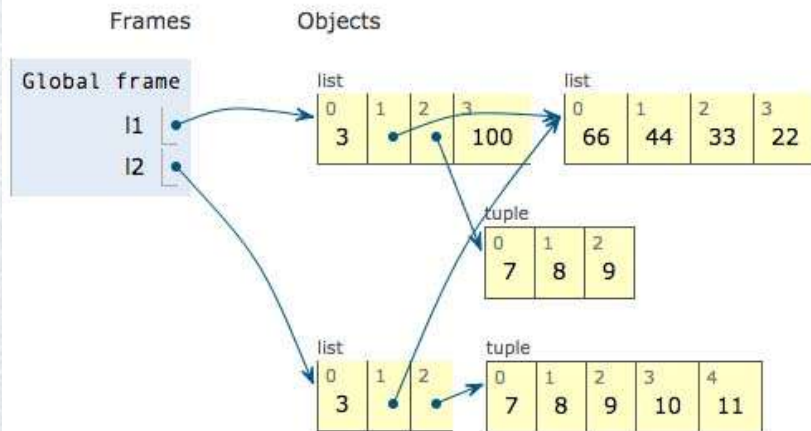
- l2 是 l1 的浅复制副本

```
l1 = [3, [66, 55, 44], (7, 8, 9)]
l2 = list(l1)    #浅复制了l1
l1.append(100)   #l1列表在尾部添加数值100
l1[1].remove(55) #移除列表中第1个索引的值
print('l1:', l1)
print('l2:', l2)
l2[1] += [33, 22] #l2列表中第1个索引做列表拼接
l2[2] += (10, 11) #l2列表中的第2个索引做元祖拼接
print('l1:', l1)
print('l2:', l2)
```

```
l1: [3, [66, 44], (7, 8, 9), 100]
l2: [3, [66, 44], (7, 8, 9)]
l1: [3, [66, 44, 33, 22], (7, 8, 9), 100]
l2: [3, [66, 44, 33, 22], (7, 8, 9, 10, 11)]
```

Print output (drag lower right corner to resize)

```
l1: [3, [66, 44], (7, 8, 9), 100]
l2: [3, [66, 44], (7, 8, 9)]
l1: [3, [66, 44, 33, 22], (7, 8, 9), 100]
l2: [3, [66, 44, 33, 22], (7, 8, 9, 10, 11)]
```





Python和Numpy进阶 知识补充

提纲



- ☐ Python基础
- ☐ NumPy与PyTorch
- ☐ 人工智能模型

索引|Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式

- 索引一个元素:

`x[1,2] → 5 with scalars`

- 使用切片 (slice) 索引一块子区域:

`x[:,1:] →`

0	1	2
3	4	5
6	7	8
9	10	11

with **slices**

`x[:, ::2] →`

0	1	2
3	4	5
6	7	8
9	10	11

with **slices**
with **steps**

Slices are **start:end:step**,
any of which can be left blank



索引|Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式

- 按条件索引:

`x[x > 9] →

10	11
----	----

 with masks`

- 用数组作为数组的索引:

`x[

0	1
---	---

,

1	2
---	---

] → [x[0,1], x[1,2]] →

1	5
---	---

 with arrays`

```
# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index]
y = y[shuffled_index]
```



索引|Numpy数组

- 更多例子

```
>>> import numpy as np
>>> x = np.arange(12).reshape((4,3))
>>> x
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> x[::-1,::-1]
array([[11, 10,  9],
       [ 8,  7,  6],
       [ 5,  4,  3],
       [ 2,  1,  0]])
>>>
```

```
>>> x
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> x[2:3, -1]
array([ 8])
>>> x[-1,-1]
11
>>> x[-1]
array([ 9, 10, 11])
>>> x[:, -1]
array([ 2,  5,  8, 11])
>>> x[2:3]
array([[6, 7, 8]])
>>>
```



Python和Numpy进阶 知识补充

提纲



- □ Python基础
- □ NumPy与PyTorch
- 人工智能模型



构建分类器的流程

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取

模型训练

- 分类损失函数
- 损失函数优化和参数调优

模型测试

- 性能评价指标



二值分类问题的评价指标

- 离线评价
 - 给定一个**测试集合** $D_t = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$
 - 给定一个待测试的模型 $f(\mathbf{x})$
 - 对比人工标签 y_i 和对应的预测值 $\text{sgn}(f(\mathbf{x}_i))$

标注标签 y_i	预测标签 $\text{sgn}(f(x_i))$
-1	+1
1	1
-1	-1
-1	+1
-1	-1
1	-1
-1	-1
1	1
-1	-1
1	1

对比



	预测为正 样本	预测为负 样本
标注为 正样本	TP (true positive)	FN (false negative)
标注为 负样本	FP (false positive)	TN (true negative)

混淆矩阵

基于混淆矩阵的评价指标

- 最容易想到的指标：预测正确率Accuracy

$$\text{Accuracy} = \frac{\text{正确预测的样本数}}{\text{总样本数}} = \frac{TP+TN}{TP+FN+FP+TN}$$

标注标签 y_i	预测标签 $\text{sgn}(f(x_i))$
-1	+1
1	1
-1	-1
-1	+1
-1	-1
1	-1
-1	-1
1	1
-1	-1
1	1



	预测为正 样本	预测为负 样本
标注为正 样本	TP (true positive)	FN (false negative)
标注为 负样本	FP (false positive)	TN (true negative)

$$\text{Accuracy} = \frac{10-3}{10} = 0.7$$



Precision/Recall的计算

$$\text{Precision} = \frac{\text{正确预测的正样本数}}{\text{预测为正例的样本数}} = \frac{TP}{TP+FP}, \text{Recall} = \frac{\text{正确预测的正样本数}}{\text{标注的正样本数}} = \frac{TP}{TP+FN}$$

标注标签 y_i	预测标签 $\text{sgn}(f(x_i))$
-1	+1
1	1
-1	-1
-1	+1
-1	-1
1	-1
-1	-1
1	1
-1	-1
1	1



	预测为正 样本	预测为负 样本
标注为 正样本	TP = 3	FN = 1
标注为 负样本	FP = 2	TN = 4

$$\text{Precision} = \frac{3}{3+2} = 0.6$$

$$\text{Recall} = \frac{3}{3+1} = 0.75$$



F1: Precision和Recall的平均数

- F1值为Precision和Recall值的调和平均数

$$- F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{1}{\frac{1}{2} \left(\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} \right)}$$

- 类比：以速度P上山，以速度R下山，平均速度F是多少？

$$- F = \frac{\text{上下山总里程}}{\text{上下山总时间}} = \frac{2S}{\frac{S}{P} + \frac{S}{R}} = \frac{2PR}{P+R}$$

- 平均速度更多取决于较慢的上山速度

- ➔ F1更加接近于Precision和Recall中较小的那个数字 (soft minimum)



F1值的计算

$$\text{Precision} = \frac{\text{正确预测的正样本数}}{\text{预测为正例的样本数}} = \frac{TP}{TP+FP}, \text{Recall} = \frac{\text{正确预测的正样本数}}{\text{标注的正样本数}} = \frac{TP}{TP+FN}$$

标注标签 y_i	预测标签 $\text{sgn}(f(x_i))$
-1	+1
1	1
-1	-1
-1	+1
-1	-1
1	-1
-1	-1
1	1
-1	-1
1	1

对比

	预测为正样本	预测为负样本
标注为正样本	TP = 3	FN = 1
标注为负样本	FP = 2	TN = 4

$$\text{Precision} = \frac{3}{3+2} = 0.6$$

$$\text{Recall} = \frac{3}{3+1} = 0.75$$

$$F1 = \frac{2 \times 0.6 \times 0.75}{0.6 + 0.75} = 0.6667$$



线性回归

- 在坐标纸上给定三个点的训练集合
 - $(x_1, y_1), (x_2, y_2), (x_3, y_3)$
- 并且规定 $f(x)$ 为线性函数（直线）
 - $f(x) = wx + b$
- 三个点，一条直线
 - 可能无法满足这条直线完美穿越所有的点（训练集合上的误差）
- 训练误差的计算
 - $\frac{1}{3}((f(x_1) - y_1)^2 + (f(x_2) - y_2)^2 + (f(x_3) - y_3)^2)$
- 模型训练要解决的问题：找到使得训练误差最小的参数组合 (w, b)



损失函数

- 使用任何一组参数都可以得到一组预测值 $\hat{y}$ ，需要一个标准来对预测结果进行度量：定量化一个目标函数式度量预测结果的好坏。
- MSE(mean square error)均方误差：线性模型应用于训练样本 x_i ，得到一组预测值 $\hat{y}_i$ ，将预测值和真实值 y_i 之间的平均的平方距离定义为损失函数：

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- N 为样本总数



损失函数

- 将线性预测函数式代入损失函数，将需要求解的参数 w 和 b 看做是损失函数 L 的自变量：

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 通过最小化损失函数求解最优参数：

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

线性回归



- 继承nn.Module, 实现线性回归

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
        super().__init__()      #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))  
  
    def forward(self, x):  
        x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
        x = x.matmul(self.w)  
        return x
```



自定义线性回归Module

- 成员函数: `forward()`, 实现前向传播过程, 其输入可以是一个或多个Tensor, 对x的任何操作也必须是Tensor支持的操作。

`forward(self, x, [y, ...])`

- 定义每次调用Module实例时执行的计算, 网络/层前向计算过程
- 所有子类/继承类都应重写此函数

```
def forward(self, x):  
    x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
    x = x.matmul(self.w)  
    return x
```

- 无需写反向传播函数, `nn.Module`能够利用`autograd`自动实现反向传播。



自定义线性回归Module

- 使用时，首先实例化一个LinearRegression类

例如：

```
def testLRmodel(in_dim, data_size = 2):  
    layer = LinearRegression(in_dim)  
    input=torch.randn(data_size,in_dim)  
    output=layer(input) #前向传播 执行forward()  
    print(output)  
    for parameter in layer.parameters():  
        print(parameter)
```

首先，实例化一个LinearRegression类

- 直观上可以将layer看成数学概念中的函数，调用model(input)即可得到input 对应的前向计算（forward）结果。

等价于layer.__call__(input).

pytorch在nn.Module中实现了__call__方法，并且在__call__方法中调用了forward函数



PyTorch线性回归

- 继承nn.Module, 实现线性回归

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
        super().__init__()      #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))  
  
    def forward(self, x):  
        x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
        x = x.matmul(self.w)  
        return x
```



PyTorch多个线性层

- Pytorch堆积多个线性层

$$z = f_1(W_1^T x + b_1)$$

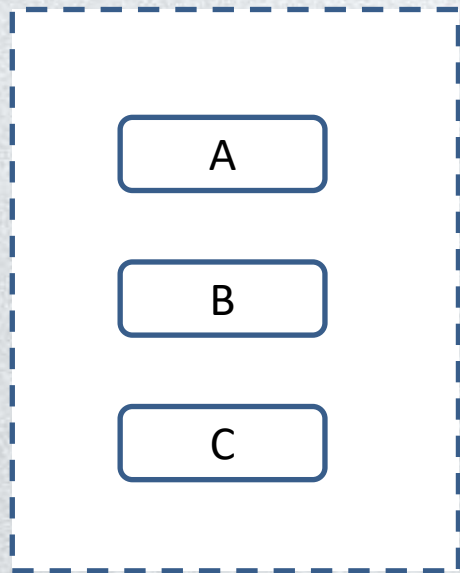
$$\hat{y} = f_2(W_2^T z + b_2)$$

```
class MLPRegression(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.linear_hidden_layer = nn.Linear(input_dim, hidden_dim, bias=True)
        self.linear_output_layer = nn.Linear(hidden_dim, 1, bias=True)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        hidden_layer_output = self.sigmoid(self.linear_hidden_layer(x))
        output = self.sigmoid(self.linear_output_layer(hidden_layer_output))
        return output
```

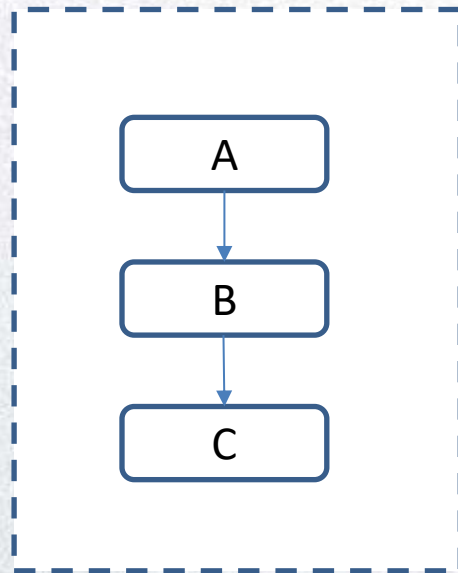
PyTorch多个线性层

- 构造函数与forward方法的“分工”：



`__init__()`

“挂载”成员变量



`forward()`

连通各模块，形成前向计算流

如何使用PyTorch训练神经网络

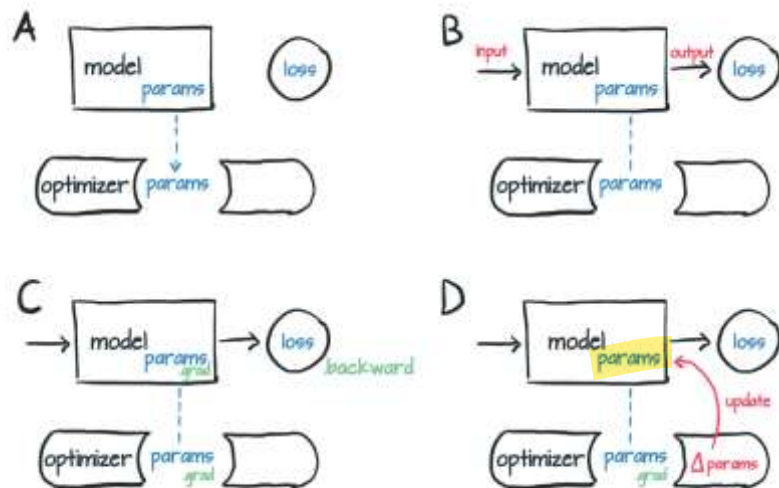


Figure 4.10 Conceptual representation of how an optimizer holds a reference to parameters (A), and after a loss is computed from inputs (B), a call to `.backward` leads to `.grad` being populated on parameter (C). At that point, the optimizer can access `.grad` and compute the parameter updates (D).

```
def train(self, x, y):
    """
    训练模型并保存参数
    输入:
        model_save_path: saved name of model
        x: 训练数据
        y: 回归真值
    返回:
        losses: 所有迭代中损失函数值
    """
    losses = []
    for epoch in range(self.epochs):
        prediction = self.model(x)
        loss = self.loss_function(prediction, y)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        losses.append(loss.item())

    if epoch % 500 == 0:
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

- 我们需要根据不同的任务选择合适的损失函数和优化器



PyTorch中的损失函数

- 常用损失函数:

- 二分类模型

- 逻辑回归损失函数: $-\frac{1}{N} \sum_{i=1}^N [y_i \log q_i + (1 - y_i) \log(1 - q_i)]$

- `nn.BCELoss`

- 二元交叉熵 (Binary Cross Entropy)

- 多分类模型

- 交叉熵损失函数:

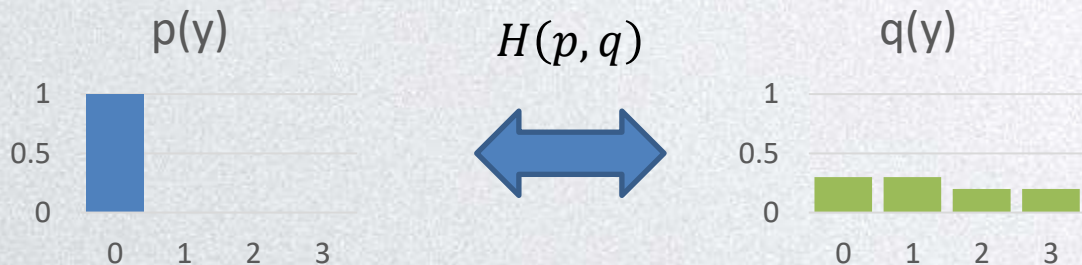
- `nn.CrossEntropyLoss`

$$\text{loss}(x, \text{class}) = -\log \left(\frac{\exp(x[\text{class}])}{\sum_j \exp(x[j])} \right)$$

PyTorch中的损失函数

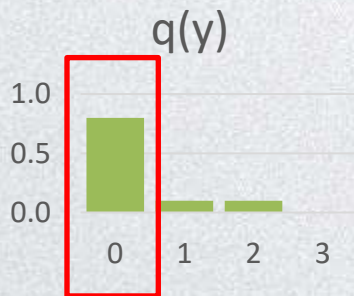
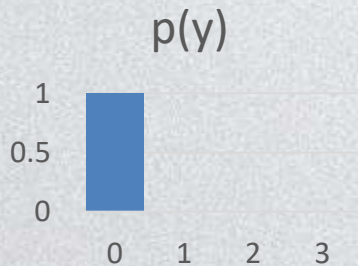
- 交叉熵:

- $H(p, q) = -\sum_y p(y) \cdot \log q(y)$
- 衡量真实分布 $p(y)$ 和预测分布 $q(y)$ 之间的差异
- 如果 $p(y) = q(y)$, 那么交叉熵 $H(p, q)$ 最小, 且刚好等于 $p(y)$ 的熵

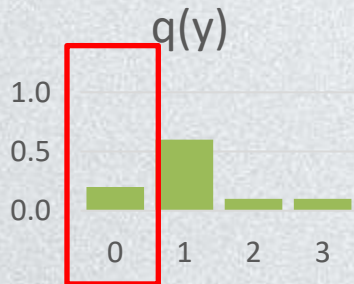


PyTorch中的损失函数

- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = -\sum_{y=0}^{C-1} p(y) \cdot \log q(y)$



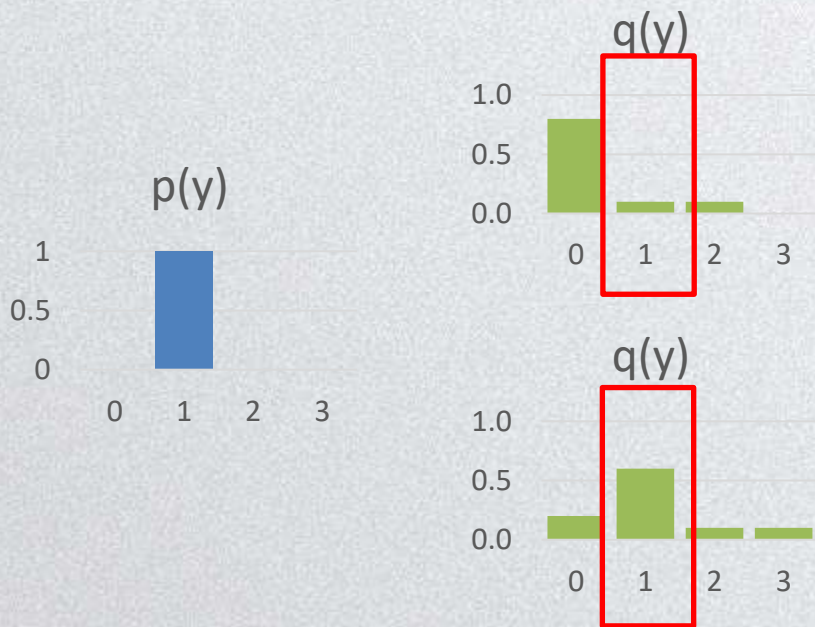
$$H(p, q) = -\log 0.8 = 0.2231$$



$$H(p, q) = -\log 0.2 = 1.6094$$

PyTorch中的损失函数

- 多分类：假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = -\sum_{y=0}^{C-1} p(y) \cdot \log q(y)$



$$H(p, q) = -\log 0.1 = 2.3026$$

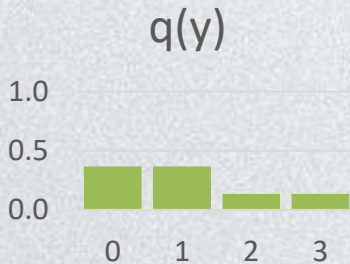
$$H(p, q) = -\log 0.6 = 0.5108$$

PyTorch中的损失函数

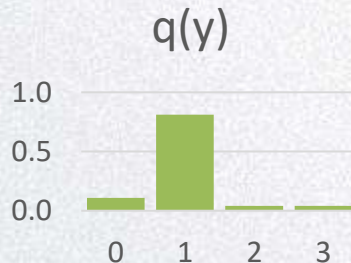
- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = -\sum_{y=0}^{C-1} p(y) \cdot \log q(y)$
- 问题： 如何得到 $q(y)$?
 - 模型输出一个C维向量 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in R^C$
 - 利用softmax函数计算:

$$q(y = i) = \frac{e^{z[i]}}{\sum_j e^{z[j]}}$$

y	z	exp(z)	q(y)
0	1.000	2.718	0.366
1	1.000	2.718	0.366
2	0.000	1.000	0.134
3	0.000	1.000	0.134



y	z	exp(z)	q(y)
0	1.000	2.718	0.110
1	3.000	20.086	0.810
2	0.000	1.000	0.040
3	0.000	1.000	0.040





PyTorch中的损失函数

- PyTorch提供的交叉熵损失函数：
 - [nn.CrossEntropyLoss](#)
 - 输入：
 - $N \times C$ 维矩阵 $\mathbf{Z}$, 其中每一行为 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in \mathbb{R}^C$
 - N 维向量 $\mathbf{y}$, 其中每个元素为 $y \in \{0, 1, 2, \dots, C - 1\}$
 - 注意：该损失函数同时计算了softmax函数和交叉熵函数：

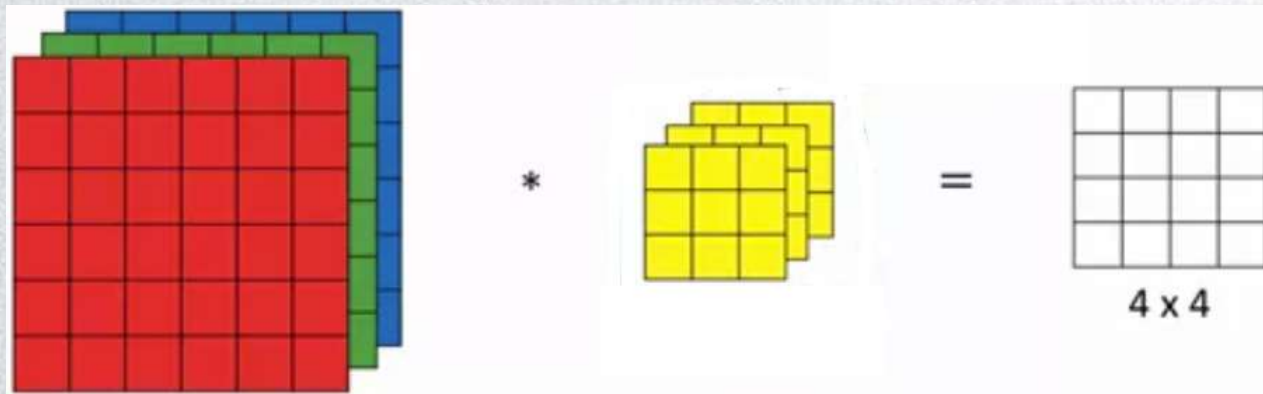
$$\text{loss}(\mathbf{Z}, \mathbf{y}) = - \sum_{i=0}^{N-1} \log\left(\frac{e^{\mathbf{Z}[i, \mathbf{y}[i]]}}{\sum_j e^{\mathbf{Z}[i, j]}}\right)$$



卷积神经网络

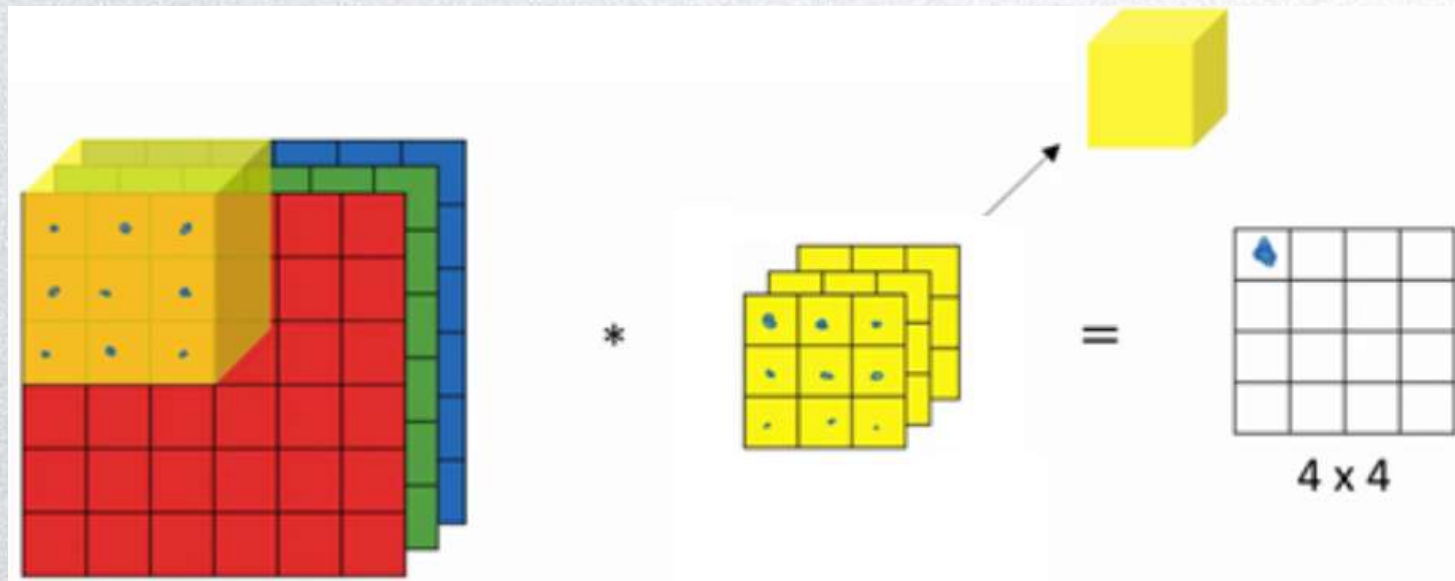
多通道卷积

- 检测RGB彩色图像的特征
- 例：彩色图像如果是 $6 \times 6 \times 3$ （高度 $\times$ 宽度 $\times$ 通道数），这里的3指的是三个颜色通道，可以把它想象成三个 6×6 灰度图像的堆叠。
- 为了检测图像的边缘或者其他特征，不是把它跟原来的 3×3 的过滤器做卷积，而是跟一个三维的过滤器，它的维度是 $3 \times 3 \times 3$ ，这样这个过滤器也有三层，对应红、绿、蓝三个通道。

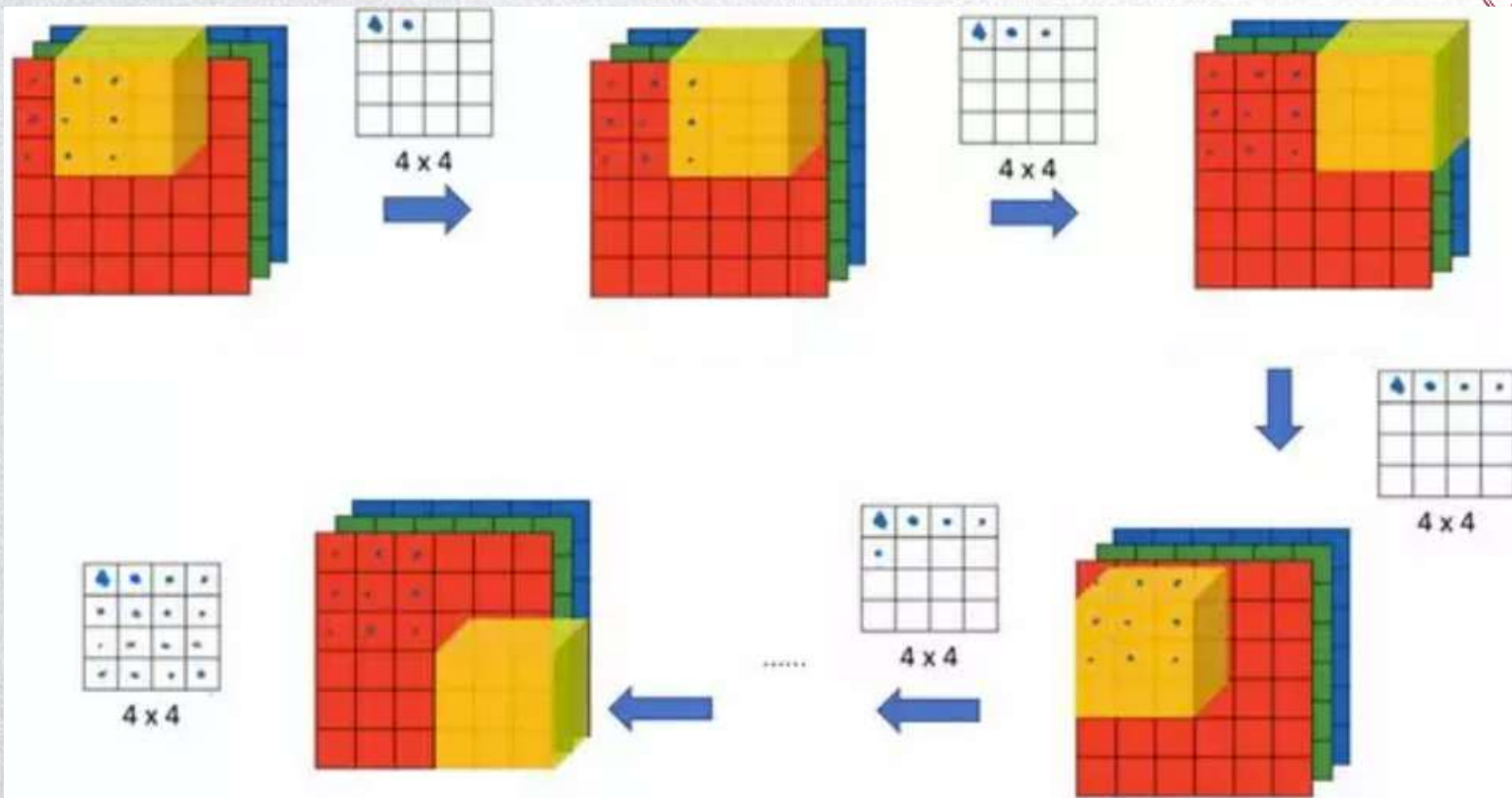


多通道卷积

- 滤波器通道数和图像通道数要匹配
- 取出27个像素值，依次和滤波器对应参数相乘，求和



多通道卷积





步长stride

- $n \times n$ 的图像，用 $f \times f$ 的滤波器进行卷积，padding= p ，步长设为 s ，
- 输出大小 $\left(\frac{n+2p-f}{s} + 1\right) \times \left(\frac{n+2p-f}{s} + 1\right)$
- 如果商不是一个整数，向下取整
- 原则：滤波必须完全处于图像中或者填充之后的图像区域内才输出相应结果

$n \times n$ image $f \times f$ filter

padding p stride s

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

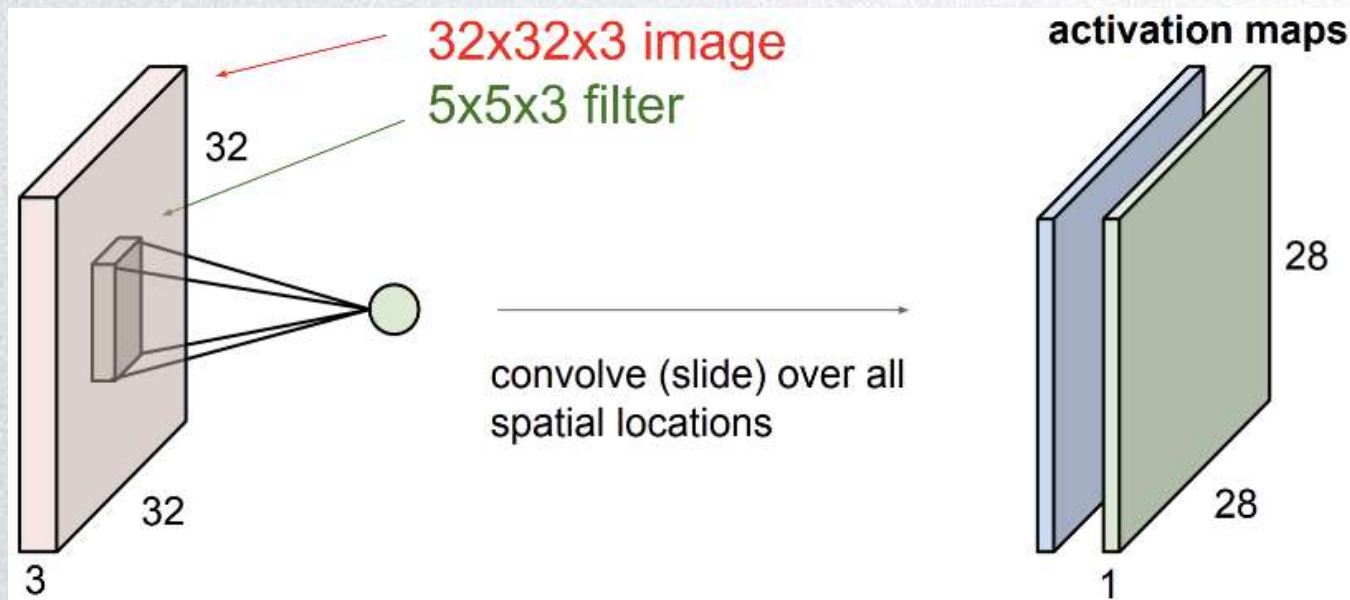


多通道卷积

- 参数的选择不同，得到不同的特征检测器
- 当输入有特定的高宽和通道数时，滤波器可以有不同的高，不同的宽，但是必须一样的通道数
- padding、stride、输出维度
- 多通道卷积：输出的通道数会等于要检测的特征数

单个卷积层

- 每个卷积核得到一个特征图





练习

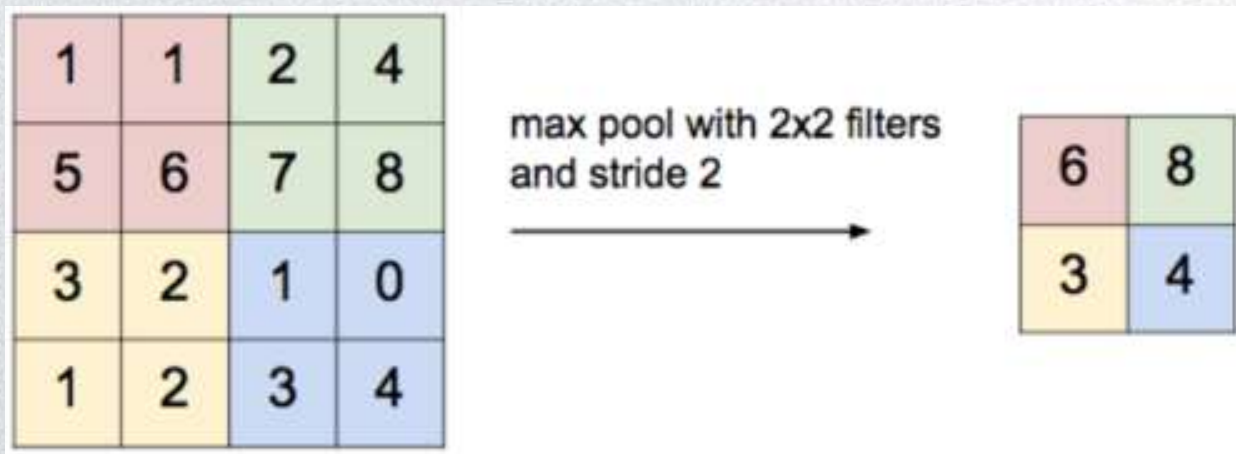
- $3 \times 32 \times 32$ 的图片，第一层用6个大小为 5×5 的滤波器， $\text{bias}=\text{True}$ ，这一层有多少个参数？

一个卷积核的参数有： $3 \times 5 \times 5 + 1 = 76$ 个参数，
一共有6个卷积核： $76 \times 6 = 456$ 个参数

- 不论输入图片有多大，参数始终都是456个。即使这些图片很大，参数却很少，这就是卷积神经网络的一个特征，可以“避免过拟合”。
- 已经知道如何提取6个特征，可以应用到大图片中，而参数数量固定不变，此例中提取一种特征只需要75个参数，相对较少。

池化层

- 特征图展开成向量可能维度很大
- 池化：缩减模型的大小；提高计算速度；提高所提取特征的鲁棒性
- Max pooling:



最大池化

- 最大池化

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

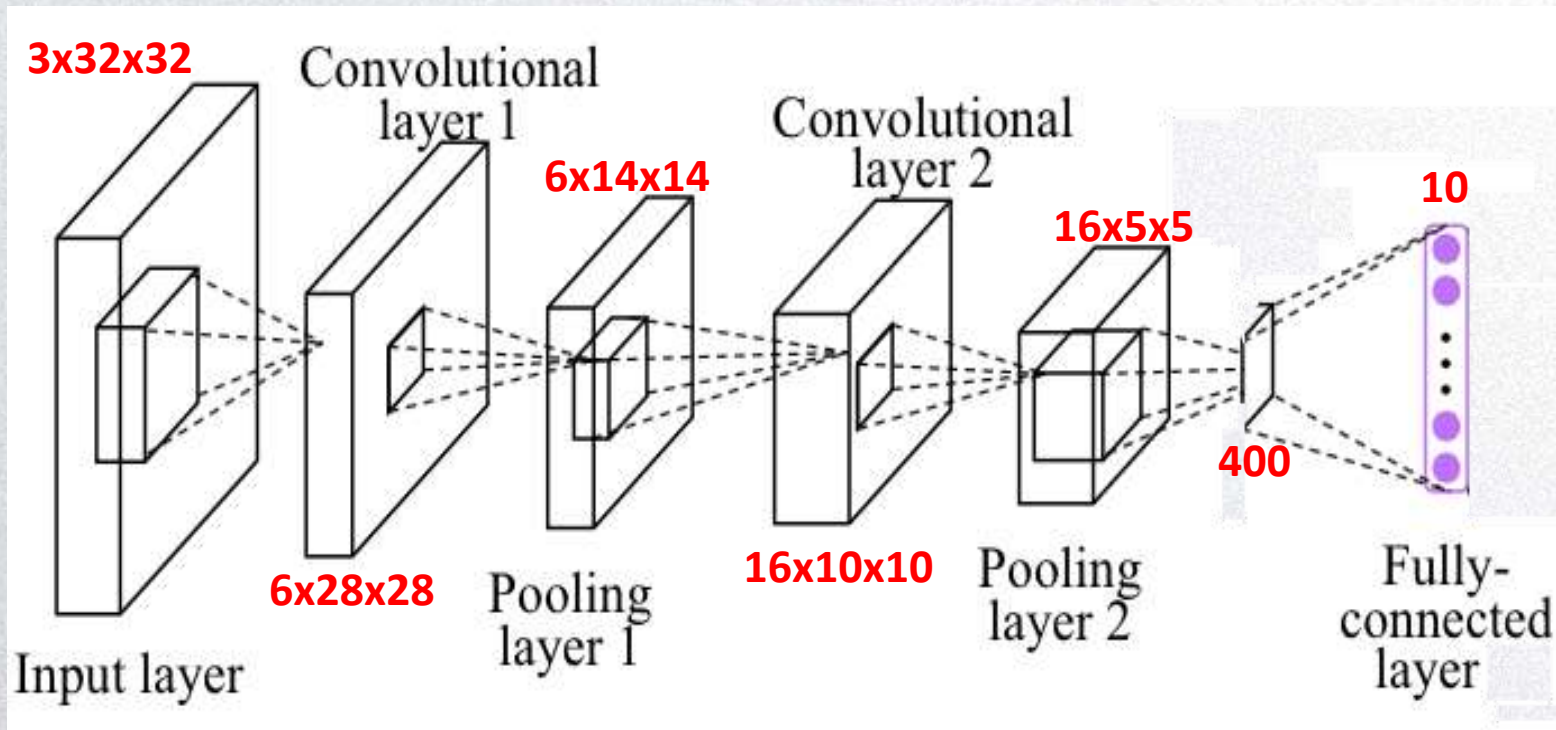


池化层总结

但是这个过滤器中没有参数

- 池化的超参数包括过滤器大小 f 、步幅 s 、池化方式（最大池化和平均池化），常用的参数值为 $f=2$ ， $s=2$ ，效果相当于高度和宽度缩减一半。
- 也可以增加表示padding的其他超参数，但很少这么用。
- 输入通道与输出通道个数相同，因为对每个通道都做了池化。
- 池化过程中没有需要学习的参数，只有这些超参数，可以是手动设置的，也可以是通过交叉验证设置的。

Practice: 计算下面过程所需要的参数



搭建卷积神经网络



```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d((2, 2), stride=2, padding=0)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d((2, 2), stride=2, padding=0)
        self.fc = nn.Linear(400, 10)
        self.nf = nn.ReLU()

    def forward(self, x):
        output1 = self.nf(self.conv1(x))
        print(output1.shape)
        output1 = self.pool1(output1)
        print(output1.shape)
        output2 = self.nf(self.conv2(output1))
        print(output2.shape)
        output2 = self.pool2(output2)
        print(output2.shape)
        # Flatten
        output3 = output2.view(1, -1)
        print(output3.shape)
        # FC
        output3 = self.fc(output3)
        print(output3.shape)
        # SoftMax
        output = torch.softmax(output3, dim=1)
        return output
```



卷积网络配置与训练

数据加载与DataLoader

数据个数声明

```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```



根据index获取对应数据样本，并返回到数据队列

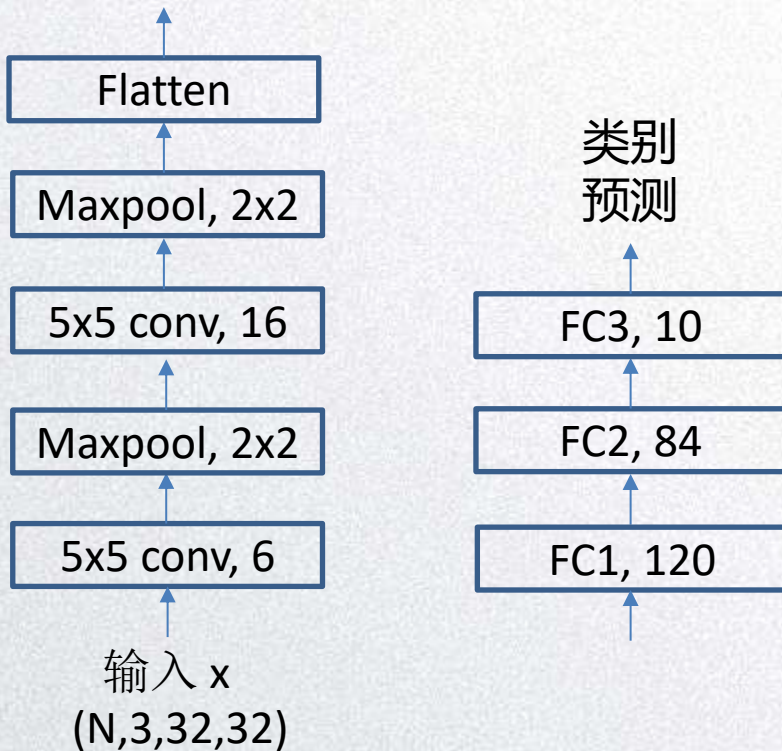
网络搭建与模型优化

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```



网络搭建与模型优化

在构造函数中，
实例化不同的
layer组件，并赋
给类成员变量

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
```

```
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```

在前馈函数中，利用实例化的
组件对网络进行搭建，并对输
入Tensor进行操作，并返回
Tensor类型的输出结果



网络搭建与模型优化

- 模型训练

全部数据
迭代次数

```
for epoch in range(2): # loop over the dataset multiple times
```

```
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
```

```
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
```

```
        # zero the parameter gradients
        optimizer.zero_grad()
```

```
        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

```
    # print statistics
```

```
    running_loss += loss.item()
```

```
    if i % 2000 == 1999: # print every 2000 mini-batches
```

```
        print('[%d, %5d] loss: %.3f' %
```

```
              (epoch + 1, i + 1, running_loss / 2000))
```

```
        running_loss = 0.0
```

```
print('Finished Training')
```

获取当前批次数据

清空模型参数的梯度

模型预测与参数优化



谢谢！